RuleDep: Modeling Complementarity and Redundancy in Rule Aggregation for KGC

Ye Sun
Beihang University
Beijing, China
sunie@buaa.edu.cn

Maribel Acosta*
Technical University of Munich
Munich, Germany
maribel.acosta@tum.de

Lei Shi*
Beihang University
Beijing, China
leishi@buaa.edu.cn

Yongxin Tong
Beihang University
Beijing, China
yxtong@buaa.edu.cn

***Abstract*—Rule-based approaches provide an interpretable and scalable alternative to embedding-based models for knowledge graph completion (KGC), but their predictive quality depends critically on how multiple fired rules are aggregated for the same candidate entity. Existing methods, from Max+ and Noisy-OR to redundancy-aware heuristics and supervised reweighting, still rely mainly on marginal rule evidence, with little explicit modeling of dependencies between co-fired rules. We observe that co-fired rules may provide either more-than-additive evidence or overlapping evidence, and therefore require explicit modeling of complementarity and redundancy during aggregation. We formulate dependency-aware aggregation in log-failure evidence space, where Noisy-OR independence gives a strong additive aggregation baseline and rule interactions appear as signed evidence gains over that baseline. Building on this formulation, RuleDep efficiently mines rule dependencies, represents pairwise complementarity or redundancy as sparse signed correction terms, and learns relation-wise aggregation weights while preserving rule-level interpretability. Experiments on seven datasets show that RuleDep improves MRR by 3.7% on average over the strongest interpretable method. The gain is especially pronounced on queries with strong complementarity but unsaturated rule evidence, where RuleDep achieves a 10.77% improvement. Meanwhile, the full RuleDep pipeline uses only 24% of the previous supervised rule aggregator's runtime on average.**

***Index Terms*—knowledge graph completion, rule aggregation, dependency modeling, interpretability, link prediction**

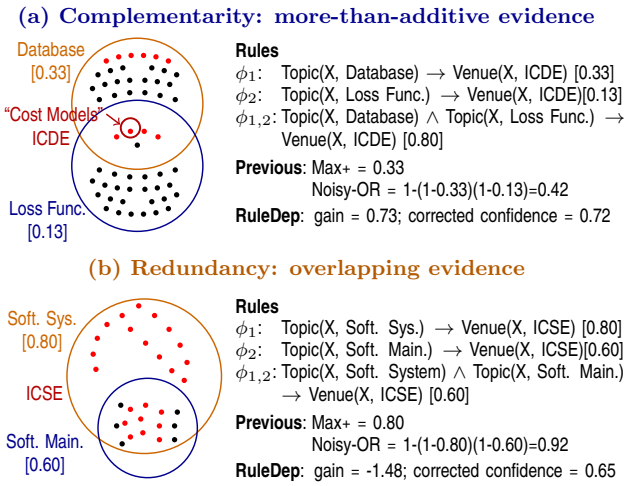


Fig. 1. An example of rule aggregation in a bibliographic KG. Each point is a paper; red and black dots are positive and negative examples for the predicted venue, respectively. **(a)** papers covered by two weak topic rules when considered separately (0.33 and 0.13) have a joint pattern with much higher confidence (0.80). **(b)** The *software system* and *software maintenance* rules for ICSE largely overlap in their covered papers.

I. Introduction

Knowledge graphs (KGs) are widely used to organize relational knowledge in domains such as biomedicine, scientific discovery, and web-scale knowledge bases. Even large KGs remain incomplete; knowledge graph completion (KGC) aims to rank plausible missing facts for queries such as $\langle h, r, ? \rangle$ or $\langle ?, r, t \rangle$. Within this broad task, rule-based KGC remains particularly attractive because it is interpretable, controllable, and scalable: each prediction can be traced back to explicit symbolic evidence rather than a latent embedding score [1]–[3]. However, the predictive quality of a rule-based system depends not only on which rules are mined, but also on how the rules fired for each candidate entity are aggregated into a single ranking. This *rule aggregation* problem, illustrated by the example in Fig. 1, is the core focus of this paper.

Existing approaches to rule aggregation fall into two broad categories. Heuristic schemes such as Max+ [2] and Noisy-OR [4] are simple and efficient but fail to capture more-than-additive evidence or over-count overlapping evidence under independence assumptions. Supervised methods, including sparse [5] and LR-Agg [3], learn rule weights or aggregation functions from data, yet their learned signal is still dominated by per-rule marginal contributions. Neither family explicitly models dependencies between rules that jointly support the same candidate entity, which leads to concrete failures when co-fired rules are not merely additive marginal evidence. Figure 1(a) shows a complementarity failure. The ICDE paper "Cost Models" [6] fires two individually weak topic rules for *database* and *loss function*, with confidences 0.33 and 0.13. Max+ therefore scores the candidate by only 0.33, and Noisy-OR raises it only to 0.42, while the composed rule combining

*Corresponding authors: Maribel Acosta and Lei Shi. Lei Shi is with School of Computer Science and Engineering, Beihang University, and the State Key Laboratory of Complex & Critical Software Environment.

Supporting materials are available at https://github.com/cs-anonymous/ruleDep.

both conditions has confidence 0.80; RuleDep's dependency correction moves the confidence to 0.72, much closer to the joint evidence. Fig. 1(b) shows that dependency is not limited to complementarity: the *software system* and *software maintenance* rules for ICSE largely overlap in their covered papers, creating the opposite failure mode where overlapping evidence is counted as independent support.

We summarize the aforementioned problem into a concrete research question: given a candidate entity and the set of standard rules that fire for it, how can rule aggregation explicitly model dependencies among co-fired rules? This is challenging for three reasons. First, interactions may occur between many pairs of rules predicting the same relation, yielding a much larger search space than ordinary rule aggregation. Second, their confidence is hard to estimate reliably, because the empirical statistics of a corresponding multi-branch rule are often sparse and noisy. Third, adding interactions risks weakening the explanatory power that makes rule-based KGC attractive.

In this paper, we propose RULEDEP, a dependency-aware aggregation framework for rule-based KGC that directly addresses the above problem. We introduce pairwise dependency features between co-fired rules, use signed gain in log-failure evidence space to identify whether a rule pair exhibits complementarity, redundancy, or near-independence, and learn trainable dependency weights that act as sparse correction terms on top of a strong rule aggregation model. Rather than replacing ordinary rule evidence, RuleDep augments it with explicit and interpretable second-order interactions.

The main contributions of this paper are as follows:

- We identify *dependency-aware rule aggregation* as a distinct problem and formulate it in log-failure evidence space, where Noisy-OR independence becomes the additive baseline and complementarity or redundancy appears as a signed correction between co-fired rules.
- We propose RULEDEP, an interpretable and scalable rule aggregation framework that operationalizes this formulation through scalable dependency mining, sparse dependency-correction learning, and relation-wise aggregation over rules and dependencies.
- We conduct an extensive evaluation on seven benchmark datasets, showing that RuleDep improves the strongest interpretable method by 3.7% and achieves the best value among interpretable methods on all 21 settings. The gain is especially pronounced on dependency-rich queries, where RuleDep achieves a 10.77% improvement.

II. RELATED WORK

A. Knowledge Graph Completion

Knowledge graph completion (KGC), also called link prediction, aims to infer missing facts in an incomplete knowledge graph. [7]–[9] The literature broadly spans three families of approaches: latent representation models, hybrid neural-symbolic methods, and interpretable symbolic rule learning.

Latent representation models dominate neural KGC by representing entities and relations as low-dimensional vectors or tensors and scoring triples via learned algebraic operations. Representative families include bilinear models (RESCAL [10], DistMult [11], ComplEx [12]), translation-based models (TransE [13], RotatE [14]), and higher-order or convolutional scoring functions (TuckER [15], ConvE [16]). Beyond triple-wise scoring functions, graph neural link prediction models propagate messages over multi-relational neighborhoods or paths, as in CompGCN [17], NBFNet [18], and AdaProp [19]. Benchmark studies [20], [21] show that properly tuned classical models remain competitive with more recent architectures. We include these models as comparison methods in Table II.

Hybrid neural-symbolic methods bring rules into latent or neural KGC pipelines in different ways. KBLRN [22] jointly optimizes latent, relational, and numerical features end-to-end. Neuro-symbolic rule learners such as Neural LP [23], DRUM [24], RNNLogic [25], RLogic [26], and their large-scale extensions [27] encode rule inference via differentiable operators or neural sequence models. GPFL [28] discovers both abstract and instantiated rules; C-NN [29] scores candidate entities using nearest-neighbour classification over rule-derived features; and embedding-guided systems exploit vector-space similarities to prune the rule search space [30], [31]. While these methods often achieve strong predictive performance, they are typically harder to interpret than purely symbolic rules and are not primarily concerned with aggregating a fixed set of mined rules.

Interpretable symbolic methods mine explicit Horn rules from the graph and remain fully transparent. The AMIE family [1], [32] represents the core ILP-based paradigm [33], refining connected Horn clauses through top-down search under language biases; extensions address exceptions [34], negative sampling [35], and confidence estimation [36]. Path-based systems such as RuleN and its successor AnyBURL [2] generalize sampled instance paths into rule templates and achieve predictive performance competitive with neural models while remaining efficient on large KGs. Related approaches explore witness paths via heuristic search (e.g., RuDiK [37]), mine frequent relational patterns (e.g., SWARM [38]), restrict the search space using ontological constraints [39]–[41], or explain link predictions with ontological closed path rules [42]. Overall, rule learning provides a rich toolbox for discovering interpretable relational patterns, but the utility of the learned rules in downstream KGC critically depends on how they are aggregated when multiple rules support the same candidate.

B. Rule Aggregation for Rule-based KGC

A central challenge in rule-based KGC is how to rank candidate entities for a query when multiple rules fire for the same candidate entity. Early systems mainly relied on heuristic aggregation, most notably Max+/lexicographic maximum and Noisy-OR [4] under an independence assumption. These strategies are simple and efficient, but are also sensitive to dependency effects: Max+ can discard more-than-additive evidence, whereas Noisy-OR can over-count overlapping evidence from highly similar explanations. Meilicke et al. [43] demonstrate that even naive rule application combined with

careful confidence calibration can yield surprisingly competitive results, highlighting the importance of score quality over aggregation complexity.

Several works therefore improve rule aggregation directly, but they differ in where they place the aggregation bias. SAFRAN [44] follows a redundancy-aware heuristic route: it clusters structurally similar rules before applying Non-Redundant Noisy-OR, reducing score inflation from overlapping evidence while preserving rule-level explanations. Supervised Knowledge Aggregation [5] instead treats aggregation itself as the learning target, introducing sparse and dense aggregators over the fired-rule set. Its sparse model learns latent rule representations with an inductive bias toward symbolic confidence aggregation, while its dense variant uses a self-attention encoder to capture richer rule-set patterns; in both cases, rule interactions are absorbed into the learned aggregator. Ott et al. [3] take a more lightweight supervised route. They analyze *confidence distortion*: pre-computed rule confidences can be biased when a rule's predictions overlap with those of other rules, because confidences are estimated rule by rule but later used inside a multi-rule aggregator. They address this by learning supervised rule confidences under learned Noisy-OR and logistic-regression aggregation models, folding overlap effects into one global marginal score per rule.

Broader probabilistic logic frameworks, including Stochastic Logic Programs [45], [46], Bayesian Logic Programs [47], and Markov Logic Networks [48], provide a useful conceptual reference because they treat rule combination as part of the probabilistic model rather than as a post-hoc scoring heuristic. In the KGC aggregation setting, however, existing methods either de-redundify rules heuristically, learn an aggregator over the fired-rule set, or calibrate per-rule marginal scores, while dependencies between concrete co-fired rules remain implicit. The remaining question is whether such dependencies can be represented explicitly as sparse, interpretable symbolic correction terms without abandoning the scalability and rule-level explanations of existing rule-based KGC pipelines.

III. Background and Problem Definition

A. Knowledge Graphs and Rules

Knowledge graphs (KGs) are collections of binary relational facts describing entities and their semantic relations. A KG is formally defined as $KG = (\mathcal{E}, \mathcal{R}, \mathcal{G})$, where $\mathcal{E}$ is the set of entities, $\mathcal{R}$ is the set of binary relations, and $\mathcal{G} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$ is the set of factual triples. Each fact is written as $\langle e_s, r, e_o \rangle$, where $e_s, e_o \in \mathcal{E}$ are the subject and object entities and $r \in \mathcal{R}$ is the relation. In rule notation, this ground fact corresponds to a ground predicate atom $r(e_s, e_o)$. Horn rules generalize such facts into relational patterns by allowing predicate atoms of the form $r(\tau_1, \tau_2)$, where each term τ_i is either a variable or an entity constant. Predicate atoms constitute the building blocks of first-order Horn rules. We write a rule ϕ as $B \to H$, where H is the head atom and $B = b_1 \wedge \cdots \wedge b_n$ is the rule body, with b_i denoting individual body atoms.

In this work, the underlying rule set is mined by AnyBURL [2], a bottom-up rule learner that generalizes sampled paths into Horn rules. Its language bias restricts ordinary prediction rules to path-shaped templates: closed-path binary rules and unary rules whose path ends with a constant or a dangling atom. Although our framework builds on AnyBURL's rule output, the aggregation stage is rule-learner agnostic and can accept rules from any system that produces Horn rules with associated confidence scores. We group AnyBURL's path rules into two canonical families as basic evidence units:

$$\begin{aligned} \phi_B: \quad & h(A_0, A_n) \leftarrow \bigwedge_{i=1}^{n} b_i(A_{i-1}, A_i), \\ \phi_U: \quad & h(A_0, c) \leftarrow \bigwedge_{i=1}^{n-1} b_i(A_{i-1}, A_i) \wedge b_n(A_{n-1}, \tau). \end{aligned} \tag{1}$$

Binary rules (ϕ_B) are *closed-path* rules: the body forms an alternative path that connects both head variables A_0 and A_n, so head and body together form a cycle. They capture relational evidence flowing between two entities through intermediate relations. In Fig. 2, rules ϕ_1–ϕ_4 are binary rules: ϕ_1 connects *paper1* to a venue via a citation path, while ϕ_3 and ϕ_4 reach venues through co-authorship chains.

Unary rules (ϕ_U) have a constant c in the head and describe *property transitions*: they predict a specific entity c based on attributes or relations of the query entity. This unified form covers AnyBURL's two unary sub-types. In a U_c rule, short for a unary rule ending with an atom containing a constant, the final term τ is a constant, so the last body atom tests for a specific property value. In a U_d rule, short for a unary rule ending with a dangling atom, τ is an unconstrained variable that appears only in the final body atom, so the body only checks for the existence of a relation. In Fig. 2, rules ϕ_5–ϕ_8 are unary U_c rules because they use specific topic constants to predict fixed venue entities. Rule $\phi_{5,6}$ falls outside this base language bias: it is a composed multi-branch rule obtained by conjoining ϕ_5 and ϕ_6 for dependency analysis.

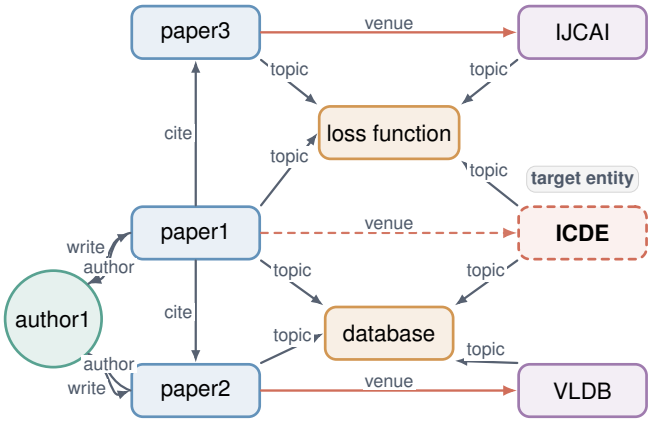
For each rule ϕ, let $\mathrm{ex}_B(\phi)$ and $\mathrm{ex}_H(\phi)$ denote the sets of groundings satisfying its body and head, respectively. The rule confidence is computed as

$$\mathrm{conf}(\phi) = \frac{\mathrm{supp}(\phi)}{|\mathrm{ex}_B(\phi)| + \lambda}, \qquad \mathrm{supp}(\phi) = |\mathrm{ex}_B(\phi) \cap \mathrm{ex}_H(\phi)|, \tag{2}$$

where λ is a smoothing parameter that accounts for unseen negative examples, default at 5. Together, binary and unary rules form the basic evidence units used by rule-based KGC.

B. Rule Aggregation for Rule-based KGC

Consider the tail query $q = \langle \textit{paper1}, \mathrm{venue}, ? \rangle$, which asks for the venue of *paper1*. Rule application first retrieves ordinary rules whose head predicate can answer the target relation; in Fig. 2(b), this gives the eight mined venue rules ϕ_1–ϕ_8. It then grounds these rules on the KG in Fig. 2(a) with *paper1* bound to the query-side variable and collects entities predicted for the open slot. Whenever a grounding satisfies a rule body and predicts candidate c, we say that the rule *fires* for the query–entity pair (q, c). This grounding step produces



(a) Local academic KG

ϕ_1 : venue(X,Y) ← cite(X,A) ∧ venue(A,Y) [0.40]
ϕ_2 : venue(X,Y) ← topic(X,A) ∧ topic(Y,A) [0.30]
ϕ_3 : venue(X,Y) ← author(X,A) ∧ author(B,A) ∧ venue(B,Y) [0.20]
ϕ_4 : venue(X,Y) ← write(A,X) ∧ write(A,B) ∧ venue(B,Y) [0.20]
ϕ_5 : venue(X, ICDE) ← topic(X, loss function) [0.13]
ϕ_6 : venue(X, ICDE) ← topic(X, database) [0.33]
ϕ_7 : venue(X, VLDB) ← topic(X, database) [0.33]
ϕ_8 : venue(X, IJCAI) ← topic(X, loss function) [0.50]
$\phi_{5,6}$: venue(X, ICDE) ← topic(X, loss function) ∧ topic(X, database) [0.80]

(b) Mined base and composed rules

Cand.	Rules	Max+	N-OR	NRNO	RuleDep
VLDB	ϕ_1–ϕ_4, ϕ_7	0.40 (2)	**0.82** (1)	0.52 (2)	0.38 (3)
IJCAI	ϕ_1, ϕ_2, ϕ_8	**0.50** (1)	0.79 (2)	**0.70** (1)	0.45 (2)
ICDE*	ϕ_2, ϕ_5, ϕ_6	0.33 (3)	0.59 (3)	0.42 (3)	**0.85** (1)

(c) Aggregation scores and rankings

Fig. 2. Running academic KG, mined base rules, and aggregation outcomes for the query $\langle paper1, \text{venue}, ?\rangle$. Citation and co-authorship paths lead to candidate entities VLDB and IJCAI for the venue slot, while topic links additionally reach ICDE. Rules ϕ_1–ϕ_4 are binary rules, and ϕ_5–ϕ_8 are unary rules ending with an atom containing a constant; these are ordinary mined rules under the AnyBURL language bias. Rule $\phi_{5,6}$ is a composed multi-branch rule rather than a base prediction rule. In the aggregation table, rankings are in parentheses, bold marks the top-ranked candidate entity, and only RuleDep ranks the ground-truth ICDE answer first by modelling the complementarity between ϕ_5 and ϕ_6. The dashed edge marks the target fact.

three candidate entities: VLDB is supported by ϕ_1–ϕ_4 and ϕ_7, IJCAI by ϕ_1, ϕ_2, ϕ_8, and the ground-truth ICDE by ϕ_2, ϕ_5, ϕ_6; the composed multi-branch rule $\phi_{5,6}$ exposes the stronger joint pattern behind ICDE but is not part of the base fired-rule set. In general, rule application produces candidate entities and, for each pair (q, c), a fired-rule set $\Phi_{q,c}$. Head queries $\langle ?, r, t\rangle$ are handled analogously through the inverse direction. Rule aggregation is the step that maps each fired-rule set $\Phi_{q,c}$ to a single score for ranking candidate entities for query q.

Rule aggregation uses a scoring function $\text{score}(q, c) = f(\Phi_{q,c})$, which maps the fired-rule set to a scalar confidence used for ranking. Common choices of f include:

- **Max+** selects the highest confidence among rules in $\Phi_{q,c}$.
- **Noisy-OR** [4] assumes conditional independence and computes $1 - \prod_{\phi \in \Phi_{q,c}} (1 - \text{conf}(\phi))$.
- **NRNO** [44] suppresses structurally similar rules before applying Noisy-OR or weighted summation.
- **Supervised variants**, such as Sparse [5] and LR-Agg [3], learn f from training data but still leave concrete rule-pair interactions implicit.

Fig. 2(c) illustrates the limitation. Max+ keeps only the strongest fired rule and therefore chooses IJCAI via ϕ_8, whereas Noisy-OR over-accumulates VLDB's larger fired-rule set. NRNO suppresses part of this redundancy but still misses ICDE. The missing signal is complementarity: ϕ_5 and ϕ_6 have low marginal confidence, yet their joint condition provides more-than-additive evidence, corresponding to the high-confidence composed multi-branch rule $\phi_{5,6}$. RuleDep captures this complementarity as an explicit dependency and correctly ranks ICDE first.

C. Dependency-Aware Rule Aggregation

The core limitation of existing aggregators is their structural inability to represent *non-additive joint evidence*—cases in which two co-fired rules jointly provide stronger or weaker support than the per-rule additive baseline predicts. As shown in Fig. 2(c), none of Max+, Noisy-OR, or NRNO can boost ICDE to rank 1, because they never score rule pairs jointly.

Although multi-branch rules can in principle represent such interactions, rule aggregation already provides a low-cost approximation to joint evidence: it combines separately mined rules that co-fire for the same candidate entity without materializing all conjunctive rules. Existing aggregators, however, approximate this evidence only coarsely, relying on marginal rule scores, independence assumptions, or generic redundancy suppression while ignoring concrete joint patterns and rule-pair interactions.

This motivates the problem of *dependency-aware rule aggregation*. For a query q and a candidate entity c, let $\Phi_{q,c}$ denote the set of fired rules, and let $D_{q,c}$ denote dependency information derived from $\Phi_{q,c}$, capturing complementarity or redundancy effects between rule pairs. The resulting aggregation function is

$$\text{score}(q, c) = f_{\text{dep}}(\Phi_{q,c}, D_{q,c}), \tag{3}$$

which outputs a real-valued score used to rank candidate entities for query q in a standard rank-based KGC setting.

The key challenges are threefold. First, the search space is large: for each relation, any pair of mined rules may interact, and enumerating all corresponding multi-branch rules would grow rapidly with the rule set size. Second, interaction strength is hard to estimate: the confidence of a multi-branch rule is computed on the intersection of its branches, which is often sparse and high-variance, so it should inform dependency discovery rather than serve directly as a prediction score. Third, dependency modelling must preserve interpretability: an explanation should show not only which rules fired, but also whether their interaction is complementary, redundant, or nearly independent. Addressing these challenges defines the scope of this work.

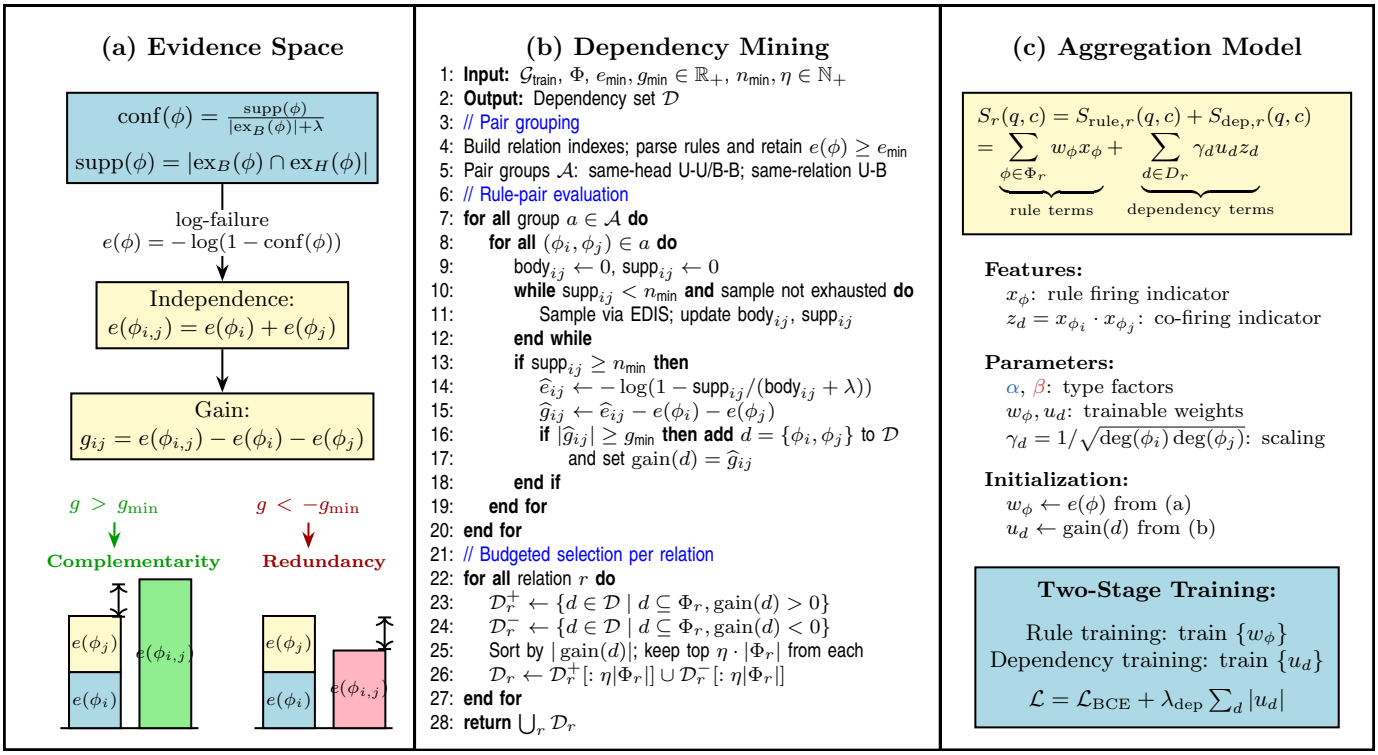


Fig. 3. Overview of the RULEDEP method. **(a) Evidence Space (Sec. IV-A):** transformation from confidence to log-failure evidence and the definition of signed evidence gain. Complementarity ($g > g_{\min}$) indicates more-than-additive evidence, redundancy ($g < -g_{\min}$) indicates overlapping evidence. **(b) Dependency Mining (Sec. IV-B):** scalable pipeline for discovering dependencies from the training graph through pair grouping, rule-pair evaluation, and budgeted selection. **(c) Aggregation Model (Secs. IV-C and IV-D):** the base linear model combines rule terms (using evidence from (a)) with dependency terms (using gain from (b)); a type-factor extension and two-stage training refine the final aggregator.

IV. METHOD

This section formalizes RULEDEP, a dependency-aware aggregation framework for rule-based KGC. As shown in Fig. 3, the method is organized around three main modules. Sec. IV-A establishes the theoretical foundation by transforming confidence to log-failure evidence space, where independence becomes additive and deviations are captured by signed evidence gain. Sec. IV-B mines dependencies from the training graph, computing gain values for dependencies that exhibit strong complementarity or redundancy. Sec. IV-C defines the base linear aggregation model over rules and dependency corrections, then adds type factors for relation-local calibration. Sec. IV-D describes the training procedure within the aggregation module, including the loss function, rule training stage, and dependency training stage.

A. Log-Failure Evidence Space

We describe the formulation for a tail query $q = \langle h, r, ? \rangle$; head queries are handled analogously through the inverse direction. Rule application generates candidate entities for the open slot, and each candidate is supported by the subset of rules that fire for it. Since only rules predicting relation r can affect this query, RuleDep works in the relation-local rule space $\Phi_r = \{\phi \mid \text{head}(\phi) = r\}$. Let $x_\phi(q,c) \in \{0,1\}$ indicate whether rule $\phi \in \Phi_r$ fires on candidate entity c for query q.

For illustration, consider two base rules $\phi_i : B_i \to H$ and $\phi_j : B_j \to H$ that share the same rule head H, where B_i and B_j are full rule bodies. After alpha-renaming non-head variables so the branches share only the head variables, let $\phi_{i,j} : B_i \wedge B_j \to H$ denote their composed multi-branch rule. The probabilistic assumption behind Noisy-OR is that the two rules fail independently to support the target fact [4]:

$$P(\neg H \mid B_i \cap B_j) = P(\neg H \mid B_i) P(\neg H \mid B_j). \tag{4}$$

To turn the product in Equation (4) into an additive form, we transform confidence into log-failure evidence, i.e., the negative log failure probability:

$$e(\phi) = -\ln\big(1 - \text{conf}(\phi)\big). \tag{5}$$

To avoid producing infinite values, we cap the resulting evidence to the range $[0, 7]$ when applying Equation (5). The support and confidence of $\phi_{i,j}$ are defined exactly as for an ordinary rule, but using the joint body $B_i \wedge B_j$. RuleDep does not add $\phi_{i,j}$ as a new prediction rule; instead, it uses $e(\phi_{i,j})$ only to measure interaction between the two base rules.

In this space, the Noisy-OR independence assumption becomes additive: if ϕ_i and ϕ_j fail independently, then $e(\phi_{i,j}) = e(\phi_i) + e(\phi_j)$. This baseline is reasonable when two rules provide unrelated evidence, for example when they are supported by disjoint relational paths or independent attributes. In real

KGs, however, co-fired rules can instead be complementary or redundant, as in Fig. 1. RuleDep therefore models interaction as a deviation from this additive baseline. We define the signed evidence gain in Equation (6),

$$g_{i,j} = e(\phi_{i,j}) - e(\phi_i) - e(\phi_j), \tag{6}$$

where $e(\phi)$ is the log-failure evidence from Equation (5). Thus, $g_{i,j}$ measures how far the joint evidence departs from the Noisy-OR baseline. RuleDep uses a minimum absolute gain threshold $g_{\min}$ to separate this departure into three regimes: pairs with $|g_{i,j}| \leq g_{\min}$ are treated as approximately independent and are not modeled as dependencies; pairs with $g_{i,j} > g_{\min}$ are complementary because their joint pattern provides more-than-additive evidence; and pairs with $g_{i,j} < -g_{\min}$ are redundant because their joint pattern provides less new evidence due to overlap. Retained dependencies will use these measured departures as structured corrections, rather than as ad hoc features or new multi-branch prediction rules.

In operationalizing this formulation, we stop at the *pairwise* level. Although higher-order interactions ($k \geq 3$) are possible in principle, pairwise dependencies offer a practical balance between expressiveness, interpretability, and computational cost. Higher-order gains would require estimating increasingly specific joint rule patterns and combining more estimated evidence terms; since the pairwise gain already takes the difference of three log-failure evidence terms, such extensions can rely on fewer joint groundings and amplify sampling variance. We therefore use pairwise dependencies as the minimal granularity at which complementarity and redundancy can be explicitly represented.

B. Scalable Dependency Mining

The signed evidence gain defined above specifies when a pair of rules should be treated as approximately independent, complementary, or redundant. We now describe how RuleDep obtains such dependencies from the training graph. The key computational challenge is that the rule learner produces symbolic rules without materializing every grounding, making exhaustive enumeration of all $O(|\Phi_r|^2)$ rule pairs prohibitive. Algorithm in Fig. 3(b) summarizes the RuleDep mining pipeline, which uses (1) pair grouping to collect only rule pairs that can co-fire for some query q and candidate entity c, (2) local rule-pair evaluation to estimate joint evidence and signed gain within each group, and (3) budgeted selection to retain only the strongest dependencies. Only rules with $e(\phi) \geq e_{\min}$ are retained before pair grouping because negligible marginal evidence rarely yields useful interactions.

Pair grouping considers two classes of rule pairs that may fire for the same pair (q, c) of query and candidate entity. The first is same-head composition: two unary rules or two binary rules are grouped only when they have exactly the same head atom, and the resulting multi-branch rule is obtained by conjoining their alpha-renamed bodies under the shared head. The second is mixed U-B composition under the same target relation: a binary rule with head $r(X, Y)$ and a unary rule with head $r(X, c)$ or $r(c, X)$ can be grouped only on groundings where the corresponding argument of the binary head is fixed to c, i.e., $Y = c$ or $X = c$, respectively. Pairs outside these two classes either predict different query relations or cannot co-fire for the same candidate entity, so they cannot affect ranking through a dependency term in Equation (7).

Given these pair groups, local rule-pair evaluation estimates their gains as follows. Same-head pairs are composed into multi-branch rules $\phi_{i,j}$, while mixed U-B pairs are aligned through co-firing statistics over the same candidate entities. The theoretical joint evidence is $e(\phi_{i,j})$, but exact materialization can generate an explosive number of groundings for binary-rule compositions. RuleDep therefore adopts AnyBURL's EDIS confidence-sampling algorithm [2], which avoids the hub-entity bias of DFS-style grounding by not over-expanding paths through a single high-degree entity; it estimates the joint evidence as $\widehat{e}_{i,j}$ and stops once the minimum support threshold $n_{\min}$ can be certified. RuleDep then computes the estimated pair-level gain $\widehat{g}_{i,j} = \widehat{e}_{i,j} - e(\phi_i) - e(\phi_j)$. If the pair is retained as a dependency $d = \{\phi_i, \phi_j\}$, we set $\mathrm{gain}(d) = \widehat{g}_{i,j}$. Thus $n_{\min}$ filters joint patterns with too few groundings, while $g_{\min}$ filters near-independent or weakly estimated interactions before they become trainable parameters. The mining hyperparameters have simple domains: $e_{\min} > 0$ and $g_{\min} > 0$ are small positive real thresholds, and $n_{\min} \in \mathbb{N}_+$ is a minimum support count.

After gain-based filtering, the remaining dependencies for each relation r are budgeted. Let $\eta \in \mathbb{N}_+$ be the integer budget multiplier. For each polarity, RuleDep retains at most $\eta \cdot |\Phi_r|$ dependencies, ranked by absolute gain, where $|\Phi_r|$ is the number of rules predicting r. This keeps the dependency set sparse and relation-proportional. We analyze the dependency-mining cost as follows. If K_a is the number of retained rules in local composition group a and C_a is the average cost of estimating one composed pair, rule-pair evaluation costs $O\left(\sum_a K_a^2 C_a\right)$, followed by linear filtering and relation-local sorting. The budget then bounds the retained dependencies for relation r by $O(\eta|\Phi_r|)$ per polarity, so the final dependency set remains linear in the number of retained rules after mining.

C. Dependency-Aware Linear Aggregation

The aggregation model takes as input a relation-specific dependency set $\mathcal{D}_r \subseteq \binom{\Phi_r}{2}$ and the gain value $\mathrm{gain}(d)$ for each dependency $d \in \mathcal{D}_r$. Together with the rule evidence from Sec. IV-A, these quantities define a relation-local linear scorer over fired rules and co-fired rule pairs.

Previous supervised rule aggregators such as LR-Agg [3] learn a score from fired-rule features, primarily estimating per-rule marginal contributions. RuleDep extends this by adding dependency terms: in log-failure evidence space, both rule contributions and dependency corrections can be linearly combined. The key design is to treat dependency as an *additive correction term*:

$$\begin{aligned} S_r(q,c) &= S_{\mathrm{rule},r}(q,c) + S_{\mathrm{dep},r}(q,c) \\ &= \sum_{\phi \in \Phi_r} w_\phi x_\phi(q,c) + \sum_{d \in \mathcal{D}_r} \gamma_d u_d z_d(q,c), \end{aligned} \tag{7}$$

As shown in Fig. 3(c), $S_{\mathrm{rule},r}(q,c)$ denotes the first summation and $S_{\mathrm{dep},r}(q,c)$ denotes the second, dependency-correction summation. We omit relation-specific intercepts because candidate entities are ranked within the same query relation; such constants do not change their ordering and only affect probability calibration. For a dependency $d = \{\phi_i, \phi_j\}$, the activation feature is the pairwise co-firing indicator $z_d(q,c) = x_{\phi_i}(q,c)\, x_{\phi_j}(q,c)$, where $x_\phi(q,c)$ indicates whether rule ϕ fires for candidate entity c under query q. Thus, a dependency is active exactly when its two endpoint rules co-fire for the same candidate entity. To prevent a single rule from being disproportionately amplified when it participates in many active dependencies, we introduce a static structural scaling factor for each dependency $d = \{\phi_i, \phi_j\}$, defined as $\gamma_d = 1/\sqrt{\deg(\phi_i)\deg(\phi_j)}$, where $\deg(\phi)$ is the number of retained dependencies in $\mathcal{D}_r$ that include ϕ.

The trainable parameters w_ϕ and u_d are initialized from statistical estimates: $w_\phi \leftarrow e(\phi)$ from Sec. IV-A, and $u_d \leftarrow \mathrm{gain}(d)$ from Sec. IV-B. This initialization lets the model start from the estimated rule evidence and the estimated pairwise deviations observed during rule mining and dependency mining. We treat u_d as a learnable parameter rather than statically fixing it to the raw empirical gain, because second-order interaction estimates are inherently noisy: taking the difference of three logged probabilities amplifies sampling noise rapidly when co-occurrence support is sparse. Thus, signed evidence gain determines which dependency terms are introduced and how they are initialized, while supervised training adjusts their final weights for downstream ranking.

The base scorer above treats all rule and dependency parameters on the same scale. In practice, different rule types have systematically different statistical properties: supervised aggregators often learn smaller weights for binary rules than for unary rules [3], while PyClause-style rule application commonly downweights U_d rules [49]. RuleDep therefore adds lightweight type factors as a local calibration layer:

$$S_r^{\mathrm{type}}(q,c) = \sum_{\phi \in \Phi_r} \alpha^r_{\tau(\phi)} w_\phi x_\phi(q,c) + \sum_{d \in \mathcal{D}_r} \beta^r_{\tau(d)} \gamma_d u_d z_d(q,c). \tag{8}$$

The function $\tau(\cdot)$ maps either a rule or a dependency to its type group under the selected granularity. For a dependency $d = \{\phi_i, \phi_j\}$, $\tau(d)$ is the unordered endpoint-type pair induced by $\tau(\phi_i)$ and $\tau(\phi_j)$. RuleDep evaluates three granularities. *Granularity RD* uses one shared rule factor and one shared dependency factor per relation. *Granularity R2D3* uses two rule groups, $\{\mathrm{B}, \mathrm{U}\}$, where U merges U_c and U_d, yielding three dependency groups, $\{\mathrm{BB}, \mathrm{UU}, \mathrm{BU}\}$. *Granularity R3D6* keeps the three rule groups $\{\mathrm{B}, \mathrm{U}_c, \mathrm{U}_d\}$, yielding six dependency groups. Thus, R2D3 and R3D6 name the number of rule and dependency groups used by the corresponding granularity.

D. Loss Function and Training

Given the relation-local linear scorer in Equation (7) and its typed extension in Equation (8), RuleDep optimizes the rule weights, dependency corrections, and type factors by supervised training. We write $S_r(q,c)$ for the active scorer below, either the base form or the typed form. Although log-failure evidence naturally suggests the Noisy-OR-style map $1-\exp(-S_r(q,c))$, the learned signed dependency corrections can make the score negative during training. We therefore map the score to a prediction probability with a sigmoid link, which is defined on the full real line and provides stable gradients:

$$\hat{y}(q,c) = \sigma\big(S_r(q,c)\big). \tag{9}$$

The base optimization objective is the binary cross-entropy:

$$\mathcal{L}_{\mathrm{BCE}} = -\frac{1}{|\mathcal{T}|} \sum_{(q,c,y)\in\mathcal{T}} \Big[y \log \hat{y}(q,c) + (1-y) \log\big(1-\hat{y}(q,c)\big) \Big], \tag{10}$$

where $\mathcal{T}$ denotes the labeled training set of query-entity pairs.

To explicitly encourage dependency sparsity and ensure that RuleDep only keeps corrections for strong interactions, we apply an L_1 regularization penalty exclusively on the trainable dependency parameters. The final objective is defined in Equation (11),

$$\mathcal{L} = \mathcal{L}_{\mathrm{BCE}} + \lambda_{\mathrm{dep}} \sum_{d \in \mathcal{D}_r} |u_d|. \tag{11}$$

The learning is structured in two stages: the rule training stage trains the rule weights w_ϕ (and α in typed variants) while fixing $u_d = 0$, establishing a stable additive baseline; the dependency training stage introduces the initialized dependency weights u_d and the dependency type factors β, then fine-tunes the entire aggregator with a reduced learning rate on the rule parameters. This two-stage approach prevents the dependency terms from destabilizing the rule baseline during early training.

V. Experiments

A. Experimental Setup

RuleDep is evaluated on seven benchmark datasets that cover both compact and large-scale rule aggregation regimes: KG20C [50], WN18RR [16], Codex-M, FB15k-237 [51], Codex-L [52], YAGO3-10 [53], and Hetionet [54]. These datasets are derived from major knowledge bases including the Microsoft Academic Graph (MAG), Freebase [55], WordNet [56], Wikidata [57], YAGO [53], and Hetionet [54], and differ substantially in the number of entities, relations, and triples, as summarized in Table I. We use the standard KGC ranking metrics MRR, Hits@1, and Hits@10. All results follow the common filtered evaluation protocol, which removes other known true triples from the ranked candidate list.

The experimental pipeline consists of four steps. *Rule mining* uses AnyBURL with a default time budget of 1000 s per dataset; this budget already recovers the common high-confidence rules used by the aggregators, while longer mining mainly adds long-tail rules and did not yield stable validation improvements in our preliminary runs. *Rule application* applies the mined rules to generate candidate entities and fired-rule sets shared by all AnyBURL-based aggregators. *Dependency mining* evaluates valid co-firing rule pairs and computes their signed evidence gains; it uses $e_{\min} = 0.05$

TABLE I
STATISTICS OF THE SEVEN BENCHMARK DATASETS. #RULE AND #DEP DENOTE THE RETAINED RULE AND DEPENDENCY TERMS USED BY THE AGGREGATION PIPELINE.

Data	Entity	Rel.	Train	Valid	Test	#Rule	#Dep
KG20C	16,362	5	48,213	3,670	3,724	154,790	168,315
WN18RR	40,943	11	86,835	3,034	3,134	76,909	45,212
Codex-M	17,050	51	185,584	10,310	10,311	518,279	883,207
FB15k-237	14,541	237	272,115	17,535	20,466	1,737,378	3,946,389
Codex-L	77,951	69	551,193	30,622	30,622	273,472	455,330
YAGO3-10	123,182	37	1,079,040	5,000	5,000	990,481	1,707,159
Hetionet	45,158	24	1,800,157	225,020	225,020	6,103,910	7,154,604

to exclude low-evidence rules from composition, $n_{\min} = 5$ to filter tiny multi-branch groundings ($n_{\min} = 2$ on KG20C and WN18RR to match their smaller rule-grounding scale), $g_{\min} = 0.01$ to retain sufficiently strong gains, and $\eta = 2$ to cap the gain-ranked dependency budget (tested in Sec. V-D). *Supervised aggregation training* follows the two-stage procedure in Sec. IV-D: it first learns the rule baseline and then fine-tunes dependency corrections; relation-wise models are trained independently, enabling parallel execution across relations and reducing runtime and GPU memory use.

We compare methods from three families and six subfamilies. *Latent* methods provide non-symbolic KGC baselines, including factorization-based embedding models (DistMult, ComplEx, ConvE, RotatE, TuckER) [11], [12], [14]–[16], which score triples from learned entity and relation vectors, and GNN models (CompGCN, NBFNet, AdaProp) [17]–[19], which propagate information over graph neighborhoods or paths. *Hybrid* methods combine rules with differentiable or neural inference, including DRUM, Neural LP, RNNLogic, Dense, and D+S [5], [23]–[25]. *Interpretable* methods expose symbolic evidence, including rule-based link prediction methods (GPFL, AMIE3, C-NN) [28], [29], [32] and AnyBURL-rule aggregators (Max+, Noisy-OR, Sparse, SAFRAN, LR-Agg) [2]–[5], [43], [44].

All compared methods use the same dataset-provided train/validation/test splits summarized in Table I. We rerun the embedding models with the KGE toolkit. For GNN, hybrid, and some interpretable baselines, we report aligned literature results when available; missing entries in Table II indicate settings without published aligned results. We use the interpretable block as the primary reference for evaluation, while latent and hybrid methods provide contextual comparisons to widely used sub-symbolic and neural baselines when aligned results are available. All AnyBURL-based aggregation methods, including Max+, Noisy-OR, Sparse, SAFRAN, LR-Agg, and RuleDep, use the same 1000 s mined rule set, so their differences reflect aggregation rather than rule mining. *RuleDep* denotes the single dependency-aware configuration with the highest macro validation MRR, applied uniformly to all relations. *RuleDep-ens* instead selects the validation-best configuration separately for each relation and reports the aggregate test metrics from these relation-wise choices.

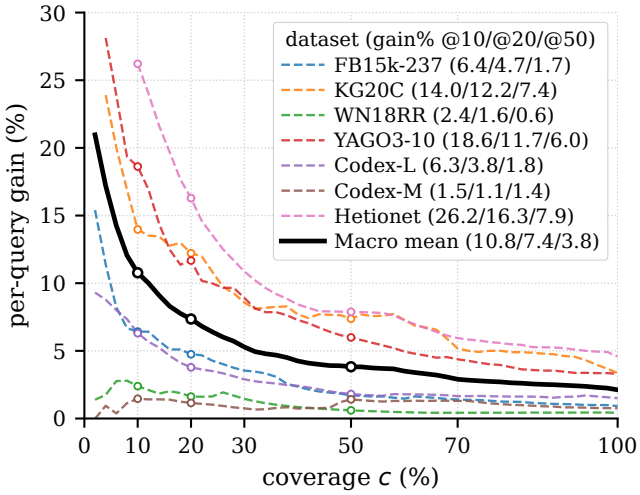


Fig. 4. Query-subset gain curves for *ratio*-3, where *ratio-k* = (*comp-k*)/(*rule-k*). The curve shows the dataset-macro subset gain as the selected top coverage fraction varies from 0 to 100.

B. Overall Results and Query-Subset Analysis

Table II reports three ranking metrics—MRR, Hits@1, and Hits@10—across seven datasets. *RuleDep-ens* leads on all 21 settings when compared against interpretable methods in Table II and leads on 15 settings when compared against all methods. On the seven-dataset average MRR, *RuleDep-ens* reaches .396, improving over the best traditional embedding baseline ComplEx (.355) by 11.5%, over Max+ (.368) by 7.6%, and over LR-Agg (.382) by 3.7%. These gains are moderate because LR-Agg already learns a relation-specific marginal weight for each fired rule. *RuleDep* therefore starts from a strong first-order rule aggregator and can only improve cases where co-fired rules provide more-than-additive or overlapping evidence.

The advantage is most visible on the larger and more heterogeneous benchmarks. On YAGO3-10, *RuleDep-ens* raises MRR from .557 with LR-Agg to .582 (+4.5%), achieves the best MRR and Hits@1 (.511), and also edges past the strongest latent and GNN baselines reported in the table. On Hetionet, where no aligned GNN or hybrid results are available, *RuleDep-ens* reaches .396 MRR, improving over LR-Agg (.371) by 6.7% and over the strongest traditional latent baseline RotatE (.257) by 54.1%.

The gains on KG20C, WN18RR, and Codex-M are smaller, which shows that dependency-aware aggregation is not uniformly beneficial across all benchmark regimes. In many queries, the rule signal is already sufficiently predictive, or the mined rule set contains few reliable co-firing patterns for the dependency layer to change the ranking. Thus the overall table averages together queries where dependencies are structurally meaningful with many queries where dependency correction has little room to operate. This motivates a finer-grained analysis over relation families and query subsets, rather than

TABLE II

OVERALL BENCHMARK COMPARISON. SYMBOLS INDICATE PUBLISHED RESULTS (†), ANYBURL WITH THE CORRESPONDING AGGREGATOR ON THE SAME MINED RULES (⋆), AND OUR CURRENT *RuleDep* PIPELINE ON THE SAME RULES (◇); UNMARKED CLASSICAL LATENT ROWS ARE RERUN ON THE SAME SPLITS, AND BLANK ENTRIES INDICATE UNAVAILABLE ALIGNED RESULTS.

	Approach	Ave.	KG20C			WN18RR			Codex-M			FB15k-237			Codex-L			YAGO 3-10			Hetio-net		
			MRR	H@1	H@10	MRR	H@1	H@10	MRR	H@1	H@10	MRR	H@1	H@10	MRR	H@1	H@10	MRR	H@1	H@10	MRR	H@1	H@10
Latent	DistMult	.333	.226	.156	.367	.452	.413	.531	.319	.246	.458	.343	.250	.531	.304	.241	.423	.501	.412	.661	.187	.128	.304
	ComplEx	.355	.203	.140	.326	.477	.438	.543	.337	.262	.476	.348	.253	.534	.294	.237	.400	.576	.505	.704	.250	.152	.470
	ConvE	.321	.171	.109	.291	.447	.411	.508	.318	.239	.464	.339	.248	.521	.303	.240	.420	.488	.399	.657	.180	.100	.318
	RotatE	.333	.212	.133	.362	.475	.426	.574	.305	.224	.454	.336	.238	.531	.250	.177	.389	.498	.405	.670	.257	.185	.403
	TuckER	–	.177	.119	.295	.459	.430	.514	.328	.259	.458	.352	.259	.536	.309	.244	.430	.544	.465	.680	–	–	–
	CompGCN†	–	–	–	–	.479	.443	.546	–	–	–	.355	.264	.535	–	–	–	.421	.392	.577	–	–	–
	NBFNet†	–	–	–	–	.551	.497	.666	–	–	–	.415	.321	**.599**	–	–	–	.550	.479	.686	–	–	–
	AdaProp†	–	–	–	–	**.562**	**.499**	**.671**	–	–	–	**.417**	**.331**	.585	–	–	–	.573	.510	.685	–	–	–
Hybrid	DRUM†	–	–	–	–	.486	.425	.586	–	–	–	.343	.255	.516	–	–	–	–	–	–	–	–	–
	Neural LP†	–	–	–	.435	.371	.566	–	–	–	.240	–	.362	–	–	–	–	–	–	–	–	–	
	RNNLogic†	–	–	–	–	.483	.446	.558	–	–	–	.344	.252	.530	–	–	–	.554	.509	.622	–	–	–
	Dense⋆	–	–	–	–	.507	.466	.587	.331	.261	.465	.335	.245	.510	–	–	–	–	–	–	–	–	–
	D+S⋆	–	–	–	–	.511	.469	.593	.342	.273	.476	.354	.267	.527	–	–	–	–	–	–	–	–	–
Interpretable	GPFL†	–	–	–	–	.480	.449	.552	–	–	–	.322	.247	.504	–	–	–	–	–	–	–	–	–
	AMIE3†	–	–	–	–	.450	.419	.514	.263	.198	.395	.247	.178	.394	.026	–	.029	.250	.206	.343	–	–	–
	C-NN†	–	–	–	–	.469	.444	.519	–	–	–	.296	.222	.446	–	–	–	.512	.424	.678	–	–	–
	Max+⋆	.368	.215	.143	.361	.497	.458	.574	.319	.249	.455	.329	.245	.503	.311	.254	.425	.554	.487	.679	.351	.292	.467
	Noisy-OR⋆	.352	.230	.157	.376	.459	.407	.567	.301	.226	.448	.338	.252	.507	.303	.242	.424	.530	.450	.675	.303	.240	.432
	Sparse⋆	–	–	–	–	.499	.459	.574	.335	.266	.467	.352	.266	.526	–	–	–	–	–	–	–	–	–
	SAFRAN⋆	–	–	–	–	.502	.459	.578	.320	.253	.449	.351	.269	.513	–	–	–	–	–	–	–	–	–
	LR-Agg◇	.382	.227	.154	.367	.500	.457	.584	.342	.274	.476	.350	.264	.524	.329	.269	.444	.557	.484	.692	.371	.308	.495
	RuleDep◇	.391	.234	.161	.373	.502	.460	.584	.345	.276	.479	.355	.267	.531	.334	.274	**.449**	.579	.507	**.707**	.390	.331	.509
	RuleDep-ens◇	.396	**.240**	**.166**	**.382**	.504	.462	.586	**.350**	**.284**	**.480**	.363	.277	.534	**.337**	**.278**	**.449**	**.582**	**.511**	**.707**	**.396**	**.337**	**.511**

TABLE III

CRITERION COMPARISON FOR QUERY-SUBSET EVALUATION. ENTRIES REPORT DATASET-MACRO RELATIVE MRR GAINS (%) AT SELECTED COVERAGE LEVELS; ARROWS INDICATE THE SORTING DIRECTION.

Criterion	gain@10	gain@20	gain@50	Middle 80% Range
$comp$-1 (↑)	6.69	5.18	3.66	$[1.68, 5.72]$
$comp$-3 (↑)	7.89	6.01	3.59	$[1.74, 5.67]$
$comp$-5 (↑)	7.32	5.95	3.68	$[1.42, 5.58]$
$rule$-1 (↓)	3.71	3.06	2.58	$[1.48, 1.80]$
$rule$-3 (↓)	4.82	3.97	2.59	$[1.39, 1.78]$
$rule$-5 (↓)	6.43	4.36	2.66	$[1.34, 1.74]$
$ratio$-1 (↑)	9.99	6.58	3.90	$[2.73, 6.89]$
$ratio$-3 (↑)	**10.77**	**7.35**	3.83	$[2.59, 7.44]$
$ratio$-5 (↑)	9.19	6.46	**3.92**	$[1.90, 7.67]$

relying only on whole-dataset averages.

We therefore conduct a query-subset evaluation to measure where dependency modeling is most beneficial, summarized in Fig. 4 and Table III. To identify such regimes, we rank test queries by mining-stage features that describe how much complementary dependency evidence is available relative to ordinary marginal rule evidence. For each criterion, we report fixed coverage levels of 10%, 20%, and 50% within each dataset; the curve additionally shows the average gain over the *top-p* of queries as p varies from 0 to 100. A steeply declining curve in Fig. 4 indicates that the feature successfully identifies queries where *RuleDep* is most beneficial. Concretely, $g(q)$ is computed as

$$g(q) = \frac{\mathrm{RR}_{\mathrm{dep}}(q)}{\mathrm{RR}_{\mathrm{rule}}(q)} - 1,$$

where $\mathrm{RR}_{\mathrm{rule}}$ and $\mathrm{RR}_{\mathrm{dep}}$ are the reciprocal ranks of the gold target after the rule training stage and dependency training stage, respectively. In Table III, each criterion selects the top coverage fraction of queries in each dataset after sorting by the indicated direction (↑ or ↓), and the reported number is the dataset-macro relative MRR gain on those selected subsets. Over all 596,060 test queries, the corresponding full-coverage macro gain is $+2.12\%$ for the *RuleDep* configuration.

The subset evaluation uses query-level features computed before supervised aggregation training. For a test query q, ordinary rule application produces candidate entities together with the rules and dependency terms active on those candidates. Let $\mathcal{C}_q$ be the multiset of complementary dependencies active among co-fired rules for query q, and let $\mathcal{R}_q$ be the multiset of fired rules. The sorting features are defined from the mining-stage estimates

$$comp\text{-}k(q) = \mathrm{mean}_k\{\mathrm{gain}(d) : d \in \mathcal{C}_q\},$$
$$rule\text{-}k(q) = \mathrm{mean}_k\{e(\phi) : \phi \in \mathcal{R}_q\},$$

where mean_k denotes the mean of the largest available k values. We then define $\textit{ratio-}k = (\textit{comp-}k)/(\textit{rule-}k)$. These criteria use $e(\phi)$ and $\text{gain}(d)$ from rule and dependency mining, not the learned aggregation parameters w_ϕ and u_d; reciprocal ranks enter only in the subsequent gain calculation.

Table III reports the representative $k = 3$ setting; the $k = 1$ and $k = 5$ variants follow the same ordering and are omitted for compactness. Sorting queries by complementarity in descending order already identifies the basic pattern: the top 10% under $\textit{comp}$-3 ($\uparrow$) gains 7.89%, whereas selecting weak marginal rule evidence by $\textit{rule}$-3 ($\downarrow$) alone gives a smaller 4.82% gain. We therefore rank queries by the aligned complementarity ratio $\textit{ratio}$-3, which matches the intended use case of the dependency layer: co-fired rules show strong complementarity while marginal rule evidence is not already saturated. Under this evaluation, $\textit{ratio}$-3 ($\uparrow$) gives a dataset-macro gain of 10.77%, and the gain remains 7.35% at 20% coverage. Thus *RuleDep* is most useful when complementarity is explicit but the strongest per-rule signals are not decisive on their own; when rule evidence is already sufficiently strong, the dependency layer has less room to improve the ranking.

The curves in Fig. 4 also show that the dependency signal is dataset-specific. Hetionet, KG20C, and YAGO3-10 exhibit clear upward trends under $\textit{ratio}$-3 ($\uparrow$), indicating that complementarity is useful on the subset of queries where multiple rules interact. By contrast, WN18RR and Codex-M are nearly flat, suggesting that dependency correction rarely changes their rankings even after selecting queries with stronger complementarity ratios.

C. Runtime Analysis

Figure 5 shows the runtime breakdown. Even after dependency discovery and training are added, the full *RuleDep* pipeline uses about 24% of the LR-Agg runtime on average over the six datasets with completed LR-Agg runs. This indicates that the additional dependency layer does not offset the efficiency gained from relation-wise training and early stopping. On Hetionet's 1.8M triples, *RuleDep* completes in about $2.4\,\text{h}$ whereas LR-Agg fails to terminate within $24\,\text{h}$.

The #Rule and #Dep columns in Table I confirm the scalability intended by the mining design in Sec. IV-B. After gain filtering and budgeting, the retained dependency set stays close to linear in the rule count: the final Dep/Rule ratio ranges from 0.59 to 2.27 across datasets. Although the largest datasets contain millions of retained dependencies, dependency mining is not the dominant cost: as annotated in Fig. 5, it finishes in less than four minutes on all datasets except Hetionet, and even on Hetionet completes in 37.5 minutes for 6.10M rules and 7.15M dependencies.

The two supervised training stages have comparable runtimes. Although the retained dependency count is often larger than the rule count, dependency training starts from the rule-trained baseline and fine-tunes with early stopping, so it does not become substantially more expensive than rule training. Across all datasets, the combined supervised training time remains under two hours.

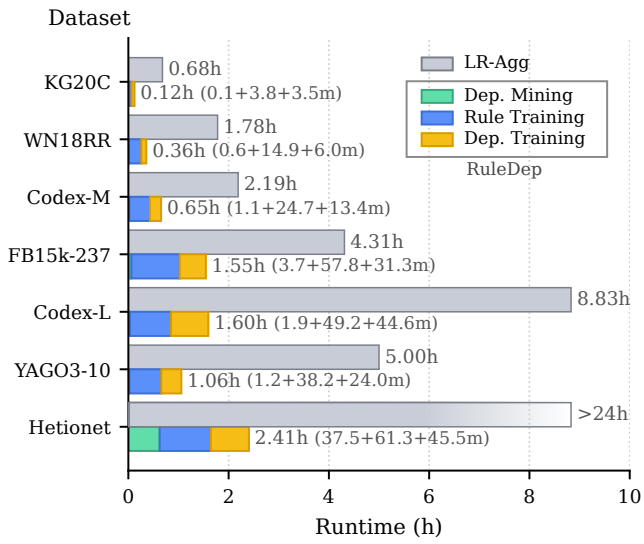


Fig. 5. Runtime comparison between LR-Agg [3] and *RuleDep* on seven datasets. The gray bar is the total runtime of LR-Agg. *RuleDep* consists of three boxed components: dependency mining (Dep. Mining), rule training (Rule Training), and dependency training (Dep. Training); parentheses report these components in the same order, in minutes. For Hetionet, the faded gray LR-Agg bar marks that LR-Agg did not finish within 24 hours.

D. Ablation and Variant Comparison

We next analyze which parts of *RuleDep* are responsible for the gains. The table entries are δMRR, i.e., $(\text{MRR}_{new} - \text{MRR}_{default})/\text{MRR}_{default}$. Table IV follows five design aspects: dependency signals (Sec. IV-A), aggregation mechanism (Sec. IV-C), training parameters, type-factor calibration, and dependency-mining budget (Sec. IV-B).

a) Dependency signals: separate the two dependency polarities. Compared with default setting, removing complementarity (w/o compl.) degrades MRR on six of seven datasets, with the largest drops on KG20C (-0.62) and WN18RR (-0.21), while Hetionet is essentially unchanged. Removing redundancy (w/o redund.) is more mixed: Codex-M and WN18RR drop, but several datasets are unaffected or slightly improve. This confirms the Sec. IV-A hypothesis that complementarity is the dominant dependency signal, while redundancy acts as a smaller selective correction.

b) Aggregation mechanism: validates the design choices in Sec. IV-C. The two largest drops in the entire ablation come from this part. *Fix-gain*, which uses raw empirical gain directly as unlearned weights, collapses on every dataset (up to -5.23 on YAGO3-10), establishing that gain alone is too noisy for discriminative ranking. *Random-init*, which discards the gain prior at initialization, drops sharply on the larger datasets (FB15k-237, YAGO3-10, Hetionet), showing that mined gain remains a meaningful structural prior even under full supervision. By contrast, *evidence-func* replaces the sigmoid link with the probability-consistent evidence map $1 - \exp(-s)$ and stays within a fraction of a point of the

TABLE IV

ABLATION RESULTS ACROSS SEVEN DATASETS. THE DEFAULT ROW REPORTS MRR; VARIANT ENTRIES REPORT δMRR IN PERCENTAGE POINTS, SO NEGATIVE VALUES INDICATE DEGRADATION.

	Approach	KG 20C	WN 18RR	Codex -M	FB15k -237	Codex -L	YAGO 3-10	Hetio -net
Dep.	default	.233	.501	.342	.348	.334	.575	.362
	w/o compl.	-0.62	-0.21	-0.07	-0.20	-0.18	-0.14	+0.02
	w/o redund.	+0.02	-0.19	-0.20	-0.01	-0.04	+0.02	+0.10
Agg.	fix-gain	-0.54	-0.31	-3.38	-4.96	-1.98	-5.23	-2.00
	random-init	+0.18	+0.07	-0.82	-3.52	-0.18	-1.25	-1.01
	evidence-func	-0.16	-0.07	-0.12	+0.03	-0.14	+0.01	+0.61
	dep-scale	-0.04	-0.13	+0.10	+0.25	+0.05	+0.52	-0.36
Train	w/o L1	-0.01	-0.20	-0.02	-0.21	-0.22	-1.40	-3.74
	sign constraint	+0.14	-0.20	-0.12	-2.83	-0.14	-0.57	-0.01
	auto-ratio	+0.13	-0.20	-0.48	+0.54	-0.70	-0.63	+0.63
Type	RD	-1.02	+0.33	+0.01	-0.81	-0.13	-0.32	+0.45
	R2D3	-1.14	-0.26	+0.09	+0.67	-0.37	+0.26	+1.55
	R3D6	-0.90	-0.27	+0.16	+0.64	-0.36	-1.33	+1.34
Budget	unconstrained	-0.02	-0.06	-0.01	-0.02	0.00	+0.00	+0.00
	1x rule	-0.01	+0.02	+0.07	-0.28	+0.00	+0.06	0.00
	4x rule	-0.02	-0.06	-0.01	-0.08	0.00	+0.00	0.00

default on every dataset. The *dep-scale* variant is also close to the default. Thus, the link function and scaling choices mainly provide numeric stability; the main signal comes from gain initialization followed by supervised calibration.

c) Training parameters: cover the loss and optimization choices used during dependency training. Removing the dependency L_1 penalty consistently reduces MRR under matched L1-enabled configurations, with the largest drops on YAGO3-10 and Hetionet. This supports the role of L_1 as a sparsity-inducing regularizer for the dependency layer. The sign-constraint variant forces all dependency weights to remain non-negative; it is mostly stable, but drops sharply on FB15k-237 (-2.83), showing that negative dependency weights are important for redundancy suppression. The *auto-ratio* variant automatically tunes the dependency-to-rule ratio per relation and remains close to the default, suggesting that the fixed $2\times$ budget is a reasonable default but not strictly optimal for every dataset. Together, the aggregation and training ablations support the core design: initialized but learnable signed dependency weights, trained with sparse regularization and without imposing a fixed sign.

d) Type calibration: tests whether type-level calibration should share parameters coarsely or finely. The optimal granularity is dataset-dependent. The coarsest *granularity RD* wins on WN18RR and Codex-L; the intermediate *granularity R2D3* is best on three datasets and yields the single largest typed gain ($+1.55$ on Hetionet); the finest *granularity R3D6* wins on KG20C and Codex-M but hurts YAGO3-10, suggesting that overly fine splits can dilute shared signal when data are insufficient. Beyond accuracy, typed weights also reveal interpretable family-level preferences: U_c-type rules dominate KG20C and the Codex datasets while U-type rules dominate the four largest datasets, with analogous patterns at the dependency-family level. Across relations within each dataset, different subsets of rule types win, so typed parameters express dataset-specific structural bias rather than a single global prior.

TABLE V

LEARNED WEIGHT BEHAVIOR BY DATASET. "NEAR0" DENOTES EFFECTIVELY ZERO WEIGHTS, "MAX" MAXIMUM ABSOLUTE VALUE, "#ACT" AVERAGE ACTIVE TERMS PER PREDICTION, AND "TOP3" THE SHARE OF ABSOLUTE CONTRIBUTION FROM THE THREE LARGEST TERMS.

	Rule				Dependency			
Data	near0	max	#act	top3	near0	max	#act	top3
KG20C	15.8%	3.06	4.28	87.0%	62.0%	0.03	0.01	100.0%
WN18RR	10.7%	4.25	13.34	75.4%	83.9%	0.47	0.13	92.3%
Codex-M	11.4%	2.99	26.69	61.7%	96.4%	0.84	0.81	87.2%
FB15k-237	15.9%	5.07	72.03	31.3%	75.6%	0.56	2.26	68.0%
Codex-L	32.0%	4.33	13.47	68.7%	83.1%	2.94	2.46	84.3%
YAGO3-10	43.8%	5.43	8.82	81.3%	98.9%	1.53	1.42	99.7%
Hetionet	34.1%	5.63	57.74	29.2%	97.2%	0.33	0.69	86.8%
Average	23.4%	4.39	28.05	62.1%	85.3%	0.96	1.11	88.3%

e) Budget: tests whether the dependency-mining budget effectively preserves useful dependencies. The default keeps at most a $2\times$ dependency-per-rule budget per polarity after gain ranking. The *unconstrained* variant removes this cap and keeps all pairs that pass the support and gain thresholds; it is virtually identical to the default on every dataset. Expanding the cap to $4\times$ also brings no measurable gain, showing that the long tail of lower-ranked dependencies contributes little after the strongest pairs have been retained. Conversely, shrinking the cap to $1\times$ remains stable on most datasets, with the largest drop only -0.28 on FB15k-237. These results indicate that useful dependencies are concentrated among the strongest gain-ranked pairs. The $2\times$ budget therefore acts mainly as a cost limiter and regularizer, keeping the dependency set sparse without bottlenecking accuracy.

E. Learned Weights and Case Study

Table V shows that the learned dependency layer is sparse and concentrated after model selection. Dependency weights have a much higher near0 value than rule weights on every dataset: dependency near0 ranges from 62.0% to 98.9%, compared with 10.7% to 43.8% for rule near0. The same pattern appears in the number of active terms per evaluated case. On average, each case contains 28.05 valid rule terms but only 1.11 valid dependency terms, indicating that most retained dependencies are suppressed by supervised training rather than used as a dense second-order layer.

The nonzero dependency corrections are nevertheless concentrated and sometimes large. The top three dependency terms account for 88.3% of dependency contribution on average, and for more than 99% on YAGO3-10. Although dependency weights are sparse, their maximum magnitude can still be comparable to rule weights on some datasets, reaching 2.94 on Codex-L and 1.53 on YAGO3-10. Thus the learned weights support the intended role of dependencies as selective corrections on top of the rule aggregation baseline.

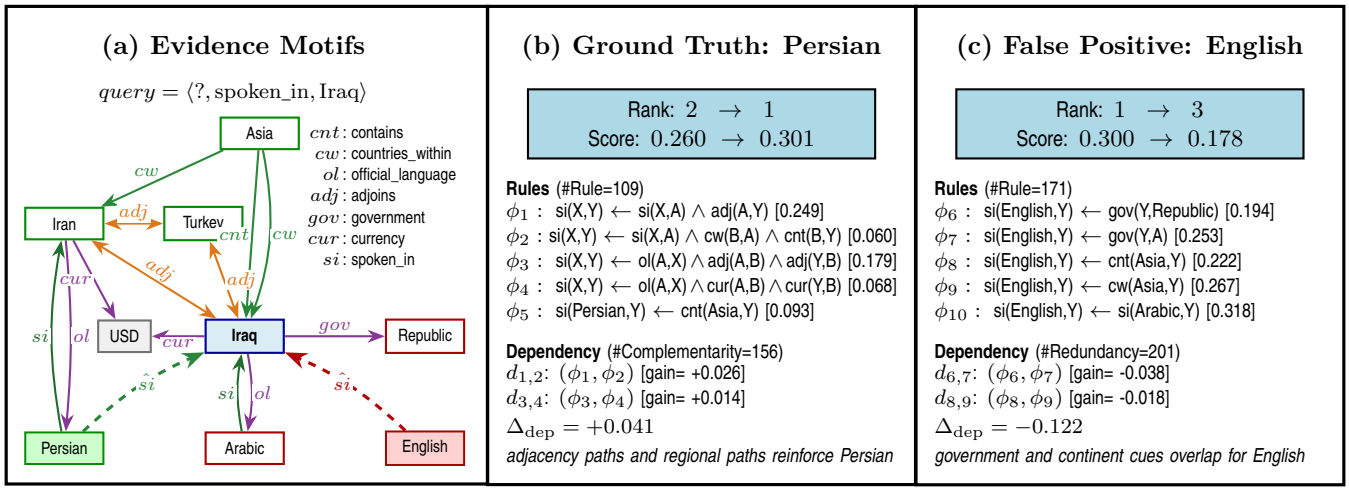


Fig. 6. FB15k-237 case study for the query "which language is spoken in Iraq?" Panel (a) shows local evidence motifs around the query. Panel (b) shows the ground-truth candidate entity Persian, whose score increases after positive dependency correction from complementary rules. Panel (c) shows the false-positive candidate entity English, whose score decreases after negative dependency correction from redundant rules. Only a small subset of the fired rules and dependencies is displayed for illustration. Δ_{dep} = accumulated dependency contribution.

As a concrete query-level example, Figure 6 shows the FB15k-237 query $q = \langle ?, \text{spoken_in}, \text{Iraq} \rangle$, i.e., which language is spoken in Iraq. Apart from Arabic, whose spoken_in edge is already present in the training graph, the held-out ground-truth answer analyzed here is Persian. The rule aggregation stage initially ranks English first (score 0.300) and the ground-truth Persian second (score 0.260). This mistake is plausible from local graph evidence: Iraq is associated with broad cues such as $\text{gov}(\text{Iraq}, \text{Republic})$, $\text{cnt}(\text{Asia}, \text{Iraq})$, and $\text{cc}(\text{Asia}, \text{Iraq})$, which activate the English rules ϕ_6–ϕ_9.

After dependency correction, Persian moves to rank 1 while English drops to rank 3. This case illustrates both complementarity promotion and redundancy suppression. Persian receives a substantial positive dependency contribution ($\Delta_{\text{dep}} = +0.041$, score $0.260 \rightarrow 0.301$). The strongest complementary pairs are (ϕ_1, ϕ_2) and (ϕ_3, ϕ_4). The first pair combines the evidence that Persian is spoken in Iran, that Iran lies in Asia, and that Iraq is adjacent to Iran while also being contained in Asia. The second pair combines $\text{ol}(\text{Iran}, \text{Persian})$ with two different but reinforcing paths, namely a shared neighboring country and the shared currency entity USD. RuleDep therefore treats these co-activated rules as complementary support for Persian.

English, by contrast, receives a negative dependency contribution ($\Delta_{\text{dep}} = -0.122$, score $0.300 \rightarrow 0.178$). The strongest negative pairs are (ϕ_6, ϕ_7) and (ϕ_8, ϕ_9), which correspond to near-duplicate government and continent cues: $\text{gov}(Y, \text{Republic})$ overlaps with the more general $\text{gov}(Y, A)$, and $\text{cnt}(\text{Asia}, Y)$ overlaps with $\text{cc}(\text{Asia}, Y)$. These rules support English only through broad regional or political regularities rather than language-specific evidence, so their co-activation is largely redundant. This case therefore shows the full role of RuleDep: it promotes the ground-truth answer Persian by rewarding complementary evidence while suppressing the false positive English by discounting redundant evidence.

Taken together, the learned-weight analysis and the case study show that the dependency layer is selective rather than diffuse. Its contribution comes from a small number of high-impact corrections on top of the rule aggregation baseline, especially when co-fired rules provide complementary evidence or redundant support that cannot be captured by marginal rule weights alone.

VI. Conclusion

We present RuleDep, a dependency-aware rule aggregation framework for rule-based knowledge graph completion. The central idea is to treat ordinary rule evidence as a strong marginal baseline and to model complementarity or redundancy between co-fired rules as an explicit pairwise correction in log-failure evidence space, preserving interpretability and scalability. Across seven datasets, RuleDep yields a mild but universal improvement over the strongest interpretable method, with a 3.7% average MRR gain and the best value among interpretable methods on all 21 settings; the gain is concentrated on dependency-rich queries, where the top 10% selected by a simple complementarity ratio reaches a dataset-macro subset gain of 10.77%. The full pipeline remains efficient, running in roughly 24% of the previous supervised rule aggregator's runtime on average. The ablations and learned weights confirm that dependency corrections are sparse and complementarity-dominated rather than uniformly spread across all rule pairs. A natural direction for future work is to extend dependency modeling beyond pairwise interactions, while controlling the sparsity, estimation noise, and computational cost of higher-order rule combinations.

REFERENCES

[1] L. A. Galárraga, C. Teflioudi, K. Hose, and F. Suchanek, "AMIE: association rule mining under incomplete evidence in ontological knowledge bases," in *Proceedings of the 22nd International Conference on World Wide Web (WWW)*, 2013, pp. 413–422.

[2] C. Meilicke, M. W. Chekol, P. Betz, M. Fink, and H. Stuckenschmidt, "Anytime bottom-up rule learning for large-scale knowledge graph completion," *VLDB Journal*, vol. 33, no. 1, pp. 131–161, 2024.

[3] S. Ott, P. Betz, D. Stepanova, M. H. Gad-Elrab, C. Meilicke, and H. Stuckenschmidt, "Rule-based knowledge graph completion with canonical models," in *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM)*, 2023, pp. 1971–1981.

[4] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, 1988.

[5] P. Betz, C. Meilicke, and H. Stuckenschmidt, "Supervised knowledge aggregation for knowledge graph completion," in *European Semantic Web Conference (ESWC)*. Springer, 2022, pp. 74–92.

[6] Y. Theodoridis, E. Stefanakis, and T. K. Sellis, "Cost models for join queries in spatial databases," in *Proceedings of the 14th International Conference on Data Engineering (ICDE)*, 1998, pp. 476–483.

[7] Q. Wang, Z. Mao, B. Wang, and L. Guo, "Knowledge graph embedding: A survey of approaches and applications," *IEEE Transactions on Knowledge and Data Engineering*, vol. 29, no. 12, pp. 2724–2743, 2017.

[8] S. Ji, S. Pan, E. Cambria, P. Marttinen, and P. S. Yu, "A survey on knowledge graphs: Representation, acquisition, and applications," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 2, pp. 494–514, 2021.

[9] H. Wu, Z. Wang, K. Wang, P. G. Omran, and J. Li, "Rule learning over knowledge graphs: a review," *Transactions on Graph Data and Knowledge*, vol. 1, no. 1, pp. 7:1–7:23, 2023.

[10] M. Nickel, V. Tresp, and H.-P. Kriegel, "A three-way model for collective learning on multi-relational data," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2011, pp. 809–816.

[11] B. Yang, W.-t. Yih, X. He, J. Gao, and L. Deng, "Embedding entities and relations for learning and inference in knowledge bases," *arXiv preprint arXiv:1412.6575*, 2014.

[12] T. Trouillon, J. Welbl, S. Riedel, É. Gaussier, and G. Bouchard, "Complex embeddings for simple link prediction," in *International Conference on Machine Learning (ICML)*. PMLR, 2016, pp. 2071–2080.

[13] A. Bordes, N. Usunier, A. Garcia-Duran, J. Weston, and O. Yakhnenko, "Translating embeddings for modeling multi-relational data," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013, pp. 2787–2795.

[14] Z. Sun, Z.-H. Deng, J.-Y. Nie, and J. Tang, "RotatE: Knowledge graph embedding by relational rotation in complex space," in *International Conference on Learning Representations (ICLR)*, 2019.

[15] I. Balažević, C. Allen, and T. Hospedales, "TuckER: Tensor factorization for knowledge graph completion," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 5185–5194.

[16] T. Dettmers, P. Minervini, P. Stenetorp, and S. Riedel, "Convolutional 2D knowledge graph embeddings," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, vol. 32, no. 1, 2018.

[17] S. Vashishth, S. Sanyal, V. Nitin, and P. Talukdar, "Composition-based multi-relational graph convolutional networks," in *International Conference on Learning Representations (ICLR)*, 2020.

[18] Z. Zhu, Z. Zhang, L.-P. Xhonneux, and J. Tang, "Neural bellman-ford networks: A general graph neural network framework for link prediction," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 29 476–29 490.

[19] Y. Zhang, Z. Zhou, Q. Yao, X. Chu, and B. Han, "AdaProp: Learning adaptive propagation for graph neural network based knowledge graph reasoning," in *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2023, pp. 3446–3457.

[20] D. Ruffinelli, S. Broscheit, and R. Gemulla, "You CAN teach an old dog new tricks! On training knowledge graph embeddings," in *International Conference on Learning Representations (ICLR)*, 2020.

[21] A. Rossi, D. Barbosa, D. Firmani, A. Matinata, and P. Merialdo, "Knowledge graph embedding for link prediction: A comparative analysis," *ACM Transactions on Knowledge Discovery from Data*, vol. 15, no. 2, pp. 1–49, 2021.

[22] A. Garcia-Duran and M. Niepert, "KBLRN: End-to-end learning of knowledge base representations with latent, relational, and numerical features," in *Proceedings of the Conference on Uncertainty in Artificial Intelligence (UAI)*, 2018, pp. 372–382.

[23] F. Yang, Z. Yang, and W. W. Cohen, "Differentiable learning of logical rules for knowledge base reasoning," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[24] A. Sadeghian, M. Armandpour, P. Ding, and D. Z. Wang, "DRUM: end-to-end differentiable rule mining on knowledge graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[25] M. Qu, J. Chen, L.-P. Xhonneux, Y. Bengio, and J. Tang, "Rnnlogic: learning logic rules for reasoning on knowledge graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[26] K. Cheng, Y. Wang, and Y. Sun, "RLogic: Recursive logical rule learning from knowledge graphs," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2022, pp. 179–189.

[27] S. Chen, Y. Cai, H. Fang, X. Huang, and M. Sun, "Differentiable neuro-symbolic reasoning on large-scale knowledge graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[28] Y. Gu, Y. Guan, and P. Missier, "Towards learning instantiated logical rules from knowledge graphs," in *European Semantic Web Conference (ESWC)*. Springer, 2020, pp. 1–16.

[29] S. Ferré, "Application of concepts of neighbours to knowledge graph completion," *Data Science*, vol. 4, no. 1, pp. 1–28, 2021.

[30] W. Zhang, B. Paudel, L. Wang, J. Chen, H. Zhu, W. Zhang, A. Bernstein, and H. Chen, "Iteratively learning embeddings and rules for knowledge graph reasoning," in *The World Wide Web Conference (WWW)*, 2019, pp. 2366–2377.

[31] P. G. Omran, K. Wang, and Z. Wang, "An embedding-based approach to rule learning in knowledge graphs," *IEEE Transactions on Knowledge and Data Engineering*, no. 4, pp. 1348–1359, 2019.

[32] J. Lajus, L. Galárraga, and F. Suchanek, "Fast and exact rule mining with AMIE 3," in *European Semantic Web Conference (ESWC)*. Springer, 2020, pp. 36–52.

[33] A. Cropper, S. Dumancic, R. Evans, and S. H. Muggleton, "Inductive logic programming at 30," *Machine Learning*, vol. 111, no. 1, pp. 147–172, 2022.

[34] M. H. Gad-Elrab, D. Stepanova, J. Urbani, and G. Weikum, "Exception-enriched rule learning from knowledge graphs," in *International Semantic Web Conference (ISWC)*. Springer, 2016, pp. 234–251.

[35] V. T. Ho, D. Stepanova, M. H. Gad-Elrab, E. Kharlamov, and G. Weikum, "Rule learning from knowledge graphs guided by embedding models," in *International Semantic Web Conference (ISWC)*. Springer, 2018, pp. 72–90.

[36] Z. Wang and J. Li, "Rdf2rules: Learning rules from rdf knowledge bases by mining frequent predicate cycles," *arXiv preprint arXiv:1512.07734*, 2015.

[37] S. Ortona, V. V. Meduri, and P. Papotti, "Rudik: Rule discovery in knowledge bases," *Proceedings of the VLDB Endowment*, no. 12, pp. 1946–1949, 2018.

[38] M. Barati, Q. Bai, and Q. Liu, "SWARM: an approach for mining semantic association rules from semantic web data," in *Pacific Rim International Conference on Artificial Intelligence (PRICAI)*. Springer, 2016, pp. 30–43.

[39] Y. Chen, S. Goldberg, D. Z. Wang, and S. S. Johri, "Ontological pathfinding," in *Proceedings of the 2016 International Conference on Management of Data (SIGMOD)*, 2016, pp. 835–846.

[40] Y. Chen, D. Z. Wang, and S. Goldberg, "Scalekb: scalable learning and inference over large knowledge bases," *VLDB Journal*, vol. 25, no. 6, pp. 893–918, 2016.

[41] G. Pirrò, "Relatedness and tbox-driven rule learning in large knowledge bases," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, vol. 34, no. 03, 2020, pp. 2975–2982.

[42] Y. Sun, L. Shi, and Y. Tong, "expath: Explaining knowledge graph link prediction with ontological closed path rules," *Proceedings of the VLDB Endowment*, vol. 18, no. 9, pp. 2818–2830, 2025.

[43] C. Meilicke, P. Betz, and H. Stuckenschmidt, "Naive rule application revisited," in *Proceedings of the Automated Knowledge Base Construction Conference (AKBC)*, 2021.

[44] S. Ott, C. Meilicke, and M. Samwald, "SAFRAN: an interpretable, rule-based link prediction method outperforming embedding models," in *Conference on Automated Knowledge Base Construction (AKBC)*, 2021.

[45] S. Muggleton, "Stochastic logic programs. advances in inductive logic programming, ed. by luc de raedt," 1995.

[46] T. Sato and Y. Kameya, "Prism: a language for symbolic-statistical modeling," in *International Joint Conference on Artificial Intelligence (IJCAI)*, vol. 97, 1997, pp. 1330–1339.

[47] K. Kersting and L. De Raedt, "Towards combining inductive logic programming with bayesian networks," in *International Conference on Inductive Logic Programming (ILP)*. Springer, 2001, pp. 118–131.

[48] M. Richardson and P. Domingos, "Markov logic networks," *Machine Learning*, vol. 62, no. 1, pp. 107–136, 2006.

[49] P. Betz, C. Meilicke, and H. Stuckenschmidt, "Rule-based knowledge graph completion," in *Reasoning Web Summer School (RW)*. Schloss Dagstuhl, 2025.

[50] H. N. Tran, T. T. Le, D. Pham, and M.-T. Nguyen, "KG20C & KG20C-QA: Scholarly knowledge graph benchmarks for link prediction and question answering," *arXiv preprint arXiv:2512.21799*, 2025.

[51] K. Toutanova and D. Chen, "Observed versus latent features for knowledge base and text inference," in *Proceedings of the Workshop on Continuous Vector Space Models and their Compositionality*, 2015, pp. 57–66.

[52] T. Safavi and D. Koutra, "Codex: A comprehensive knowledge graph completion benchmark," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 8328–8350.

[53] F. Mahdisoltani, J. Biega, and F. Suchanek, "Yago3: A knowledge base from multilingual Wikipedias," in *Proceedings of the Conference on Innovative Data Systems Research (CIDR)*, 2014.

[54] D. S. Himmelstein, A. Lizee, C. Hessler, L. Brueggeman, S. L. Chen, D. Hadley, A. Green, P. Khankhanian, and S. E. Baranzini, "Systematic integration of biomedical knowledge prioritizes drugs for repurposing," *eLife*, vol. 6, p. e26726, 2017.

[55] K. Bollacker, C. Evans, P. Paritosh, T. Sturge, and J. Taylor, "Freebase: A collaboratively created graph database for structuring human knowledge," in *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*, 2008, pp. 1247–1250.

[56] G. A. Miller, "WordNet: A lexical database for English," *Communications of the ACM*, vol. 38, no. 11, pp. 39–41, 1995.

[57] D. Vrandečić and M. Krötzsch, "Wikidata: A free collaborative knowledgebase," *Communications of the ACM*, no. 10, pp. 78–85, 2014.